

Sistema de Detección y Segmentación de Objetos en Tiempo Real mediante Ajuste Fino de YOLOv8

Trabajo Práctico Final — Deep Learning

Federico Morán

Maestría en Inteligencia Artificial

Julio de 2026

- 1 Problema y objetivo
- 2 Datos y metodología
- 3 Modelo y entrenamiento
- 4 Evaluación y resultados
- 5 Sistema en tiempo real
- 6 Conclusiones

Objetivo general

Desarrollar un sistema completo de visión por computadora que **detecte y segmente objetos en tiempo real** sobre hardware de consumo.

Objetivos específicos

- 1 Construir un conjunto de datos supervisado a partir de COCO 2017.
- 2 Ajustar un modelo de segmentación de instancias por transferencia de aprendizaje.
- 3 Evaluarlo con las métricas estándar del área (mAP e IoU, cajas y máscaras), incluida la comparación contra el modelo preentrenado de referencia.
- 4 Implementar la inferencia en tiempo real con captura de cámara web.

¿Por qué importa el tiempo real?

Robótica, asistencia, control de inventario y análisis de video imponen un compromiso exigente entre **precisión** y **latencia**, en particular sin aceleración dedicada.

Detección

Localiza cada objeto con una *caja delimitadora* y le asigna una clase.

Segmentación de instancias

Delimita, *píxel a píxel*, la silueta exacta de cada objeto individual.

Clave

YOLOv8-seg resuelve **ambas** tareas con una única pasada hacia adelante: la cabeza predice caja, clase y coeficientes de máscara simultáneamente.

Verificación: anotaciones reconstruidas desde los archivos YOLO exportados



Anotaciones de instancia reconstruidas desde los archivos YOLO exportados: cada objeto tiene caja y polígono.

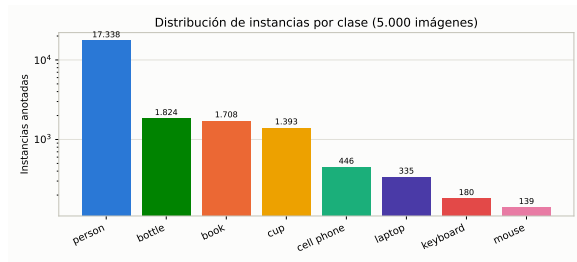
Alcance: ocho clases de entorno doméstico y de oficina

person cell phone cup
bottle laptop keyboard
mouse book

Criterio de selección

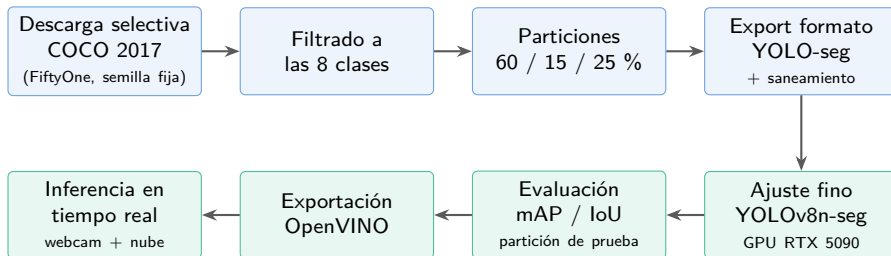
Todas pueden **exhibirse físicamente** ante una cámara web durante la demostración en vivo. El criterio es de aplicación, no de conveniencia estadística.

Los objetos de las restantes 72 categorías de COCO pasan a formar parte del **fondo**, lo que define con precisión el alcance del problema.



Desbalance pronunciado, heredado de la estadística natural de COCO: **person** concentra el 74,2 % de las instancias; **mouse** reúne 139.

Pipeline de extremo a extremo



Todo el pipeline está gobernado por **semillas fijas** (selección de datos, particiones, entrenamiento) y es reproducible de extremo a extremo.

En lugar de descargar COCO completo (~19 GB), se descargan solo las imágenes que contienen al menos una instancia de las clases objetivo.

```
dataset = foz.load_zoo_dataset(  
    'coco-2017', split='train',  
    label_types=['segmentations'],  
    classes=CLASSES,           # las 8 clases de interés  
    max_samples=5000,  
    shuffle=True, seed=42,     # descarga reproducible  
)  
view = dataset.filter_labels('ground_truth', F('label').is_in(CLASSES))
```

- **5.000** imágenes
- **23.363** instancias anotadas
- 4,67 objetos/imagen (mediana 3, máx. 47)
- Anotación a nivel de instancia (polígonos)
- Habilita detección y segmentación

Un riesgo real, detectado y corregido

FiftyOne etiqueta cada muestra descargada con su *split de origen* (train, por venir de COCO train2017). Esa etiqueta **colisiona** con la de la partición propia: `match_tags('train')` devuelve las 5.000 imágenes, no las 3.000 previstas.

```
dataset.untag_samples(['train', 'validation', 'test']) # 1. limpiar etiquetas de origen
four.random_split(view, SPLITS, seed=42)             # 2. partición propia
shutil.rmtree(DATA_DIR)                             # 3. el export FUSIONA por defecto
```

Verificación sobre los archivos, no sobre las etiquetas

El control definitivo se hace sobre lo que el entrenador *realmente lee del disco*: las tres particiones deben ser **disjuntas** y medir **3.000 / 750 / 1.250**. Un `assert` lo comprueba y detiene la corrida si falla.

Particiones disjuntas a nivel de imagen

- **Entrenamiento** — 60 %, 3.000 imágenes
- **Validación** — 15 %, 750 imágenes
monitoreo y selección del punto de control
- **Prueba** — 25 %, 1.250 imágenes
reservada; no interviene en ninguna decisión

Conversión (dos pasos)

- 1 Máscaras binarias → polígonos (tolerancia de simplificación: 2 px).
- 2 Saneamiento: se descartan los polígonos de menos de tres vértices (~ 3 % de las líneas), que Ultralytics rechaza.

Verificación: anotaciones reconstruidas desde los archivos YOLO exportados



Control de calidad: las anotaciones se reconstruyen desde los archivos YOLO exportados y se superponen sobre las imágenes. Valida la cadena completa: formato, normalización de coordenadas y asignación de índices de clase.

Modelo de segmentación de instancias de **una sola etapa** y **libre de anclas**.

- **Backbone** — CSPDarknet con bloques C2f; extrae mapas de características a múltiples escalas.
- **Neck** — PAN-FPN; fusiona semántica de alto nivel con detalle espacial de bajo nivel.
- **Cabeza desacoplada** — predice caja, clase y un vector de *coeficientes de máscara*; una rama paralela genera *prototipos* de máscara (esquema derivado de YOLACT).

La máscara de cada instancia es una **combinación lineal de los prototipos** $\Rightarrow$ costo de segmentación casi constante por imagen.

¿Por qué la variante *nano*?

3,26 M de parámetros, 11,4 GFLOPs. Es la de menor capacidad de la familia.

El requisito de tiempo real debe satisfacerse sobre la **CPU de una computadora personal**: es la variante con mejor relación desempeño-latencia para ese escenario.

Transferencia de aprendizaje: qué se hereda y qué no

Se parte de los pesos preentrenados en COCO completo. Pero el vocabulario cambia de 80 a 8 clases, y eso tiene una consecuencia que conviene explicitar:

```
Overriding model.yaml nc=80 with nc=8
```

```
Transferred 381/417 items from pretrained weights
```

Se hereda

Backbone y neck completos: las representaciones visuales aprendidas sobre las 118.000 imágenes de COCO.

Se reinicializa

36 tensores (~ 372.000 parámetros): **toda la rama de clasificación de la cabeza** (`model.22.cv3`), cuyas dimensiones dependen de `nc`.

Consecuencia de diseño

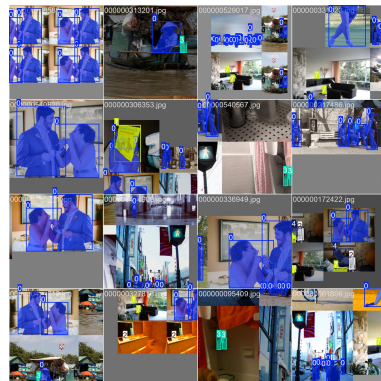
El clasificador arranca desde cero. Como el dominio de origen y el de destino **coinciden** (las 8 clases pertenecen a COCO), la estrategia es **congelar el backbone** y concentrar el presupuesto de optimización en la cabeza. Volveremos sobre esto en los resultados.

Preprocesamiento y aumento de datos

Preprocesamiento (entrenamiento e inferencia): redimensionado a 640×640 por *letterbox* (preserva la relación de aspecto, rellena con bordes neutros); intensidades normalizadas a $[0, 1]$. **Aumento** (solo entrenamiento): transforma de manera consistente las imágenes y sus polígonos.

Transformación	Valor	Justificación
<i>Mosaic</i>	1,0	Contextos y escalas atípicos
Volteo horizontal	$p=0,5$	Sin quiralidad relevante
Traslación	$\pm 10\%$	Encuadres imperfectos
Escala	$\pm 50\%$	Distancia objeto-cámara
HSV	0,015/0,7/0,4	Iluminación y color

Mosaic se desactiva en las últimas 10 épocas (`close_mosaic`). Se **omiten** volteo vertical, rotaciones y *mixup*: generarían poses inverosímiles para el dominio.



Parámetro	Valor
Modelo base	yolov8n-seg.pt (preentrenado en COCO)
Capas congeladas	freeze=10 (backbone); se entrenan neck y cabeza
Épocas	150 (early stopping, paciencia 50)
Resolución de entrada	640 × 640 (letterbox)
Tamaño de lote	32
Procesos de precarga	4
Optimizador	AdamW (selección automática), $lr_0 = 8,33 \times 10^{-4}$, momento 0,9
Decaimiento de pesos	5×10^{-4}
Precisión mixta (AMP)	Sí
Semilla	42 (modo determinista)
Hardware	GPU NVIDIA RTX 5090 (32 GB)
Duración	132 minutos (se detuvo en la época 140)

El lote y los procesos de precarga se fijaron explícitamente para acotar el consumo de memoria en un servidor compartido. El punto de control se selecciona por la **métrica de aptitud sobre validación**.

IoU — solapamiento entre la predicción P y la anotación G :

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|}$$

Se aplica **tanto a cajas como a máscaras**. Una predicción es verdadero positivo si su IoU con una anotación de la misma clase supera un umbral.

AP — área bajo la curva precisión–recall de una clase.

mAP — promedio de las AP sobre las clases.

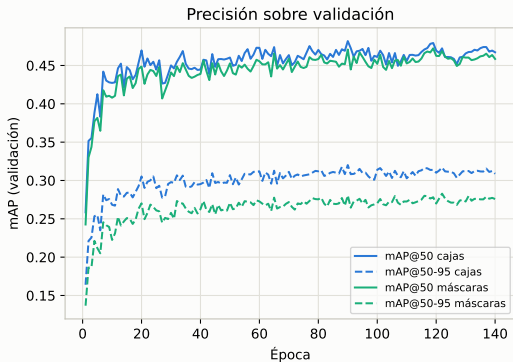
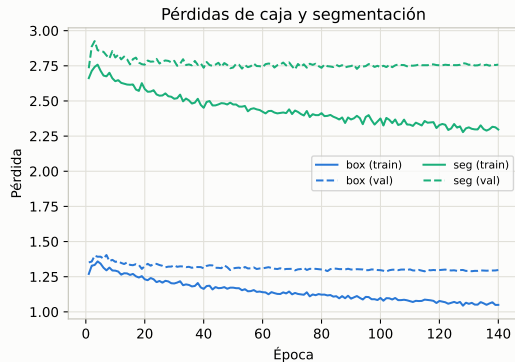
Las dos variantes estándar

mAP@50 — umbral de IoU de 0,5. Criterio laxo: mide si el objeto fue encontrado.

mAP@50-95 — promedio sobre umbrales de 0,5 a 0,95 en pasos de 0,05. Criterio estricto: premia la **precisión de la localización**. Es la métrica principal de este trabajo.

Todo se calcula por separado para **cajas** (detección) y **máscaras** (segmentación), sobre la partición de prueba (1.250 imágenes, 7.117 instancias), reservada de principio a fin.

Curvas de aprendizaje



- Las pérdidas de caja y segmentación decrecen de forma sostenida, sin divergencia: no hay sobreajuste.
- El mAP de validación crece rápido y se estabiliza en una meseta (mAP@50-95 de máscaras hasta 0,283), coherente con un modelo de *backbone* congelado.
- La pérdida de validación se ubica **por encima** de la de entrenamiento, como corresponde a la brecha de generalización sobre una partición reservada.

Métrica	Cajas (detección)	Máscaras (segmentación)
Precisión media	0,577	0,559
Recall medio	0,397	0,383
mAP@50	0,440	0,420
mAP@50-95	0,292	0,246

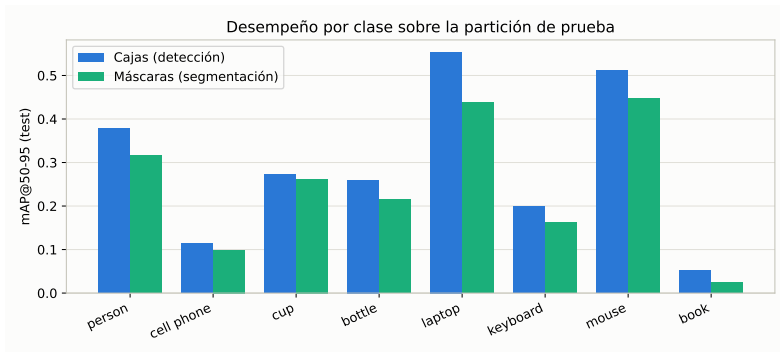
El recall es el cuello de botella

La precisión (0,577) supera con holgura al recall (0,397): el modelo acierta cuando detecta, pero **omite** muchas instancias.

Máscaras por debajo de cajas

El IoU a nivel de píxel es más estricto que el de rectángulos, sobre todo en contornos complejos como los de person.

Estos valores se interpretan frente al baseline en la comparación imparcial (dos láminas más adelante).



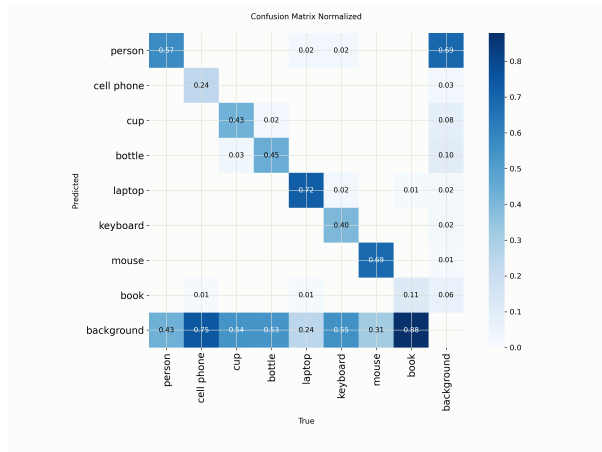
Mejores clases: laptop, mouse

mAP@50-95 de cajas de 0,554 y 0,512. Objetos rígidos, de contorno regular, poco ocluidos.

Peor clase: book

En COCO aparecen apilados o en estanterías: instancias pequeñas, con límites ambiguos y anotación inconsistente.

Matriz de confusión: ¿dónde falla el modelo?



Hallazgo

El modelo **prácticamente no confunde clases entre sí** (máximo 0,03). El modo de error dominante es la **omisión**: objetos reales clasificados como fondo.

- El vocabulario reducido hace que la *clasificación* sea sencilla.
 - El desafío restante es la *detección* de instancias pequeñas u ocluidas.
- ⇒ El margen de mejora está en el **recall**, no en la discriminación.

Omisión por clase: book 88 %, cell phone 75 %, keyboard 55 %, cup 54 %, bottle 53 %.

El sesgo oculto en la comparación con el baseline

Comparar el modelo ajustado (0,246 de máscaras) contra el preentrenado sobre nuestra partición de prueba sugiere que el preentrenado gana (0,342). **Pero esa comparación es tramposa.**

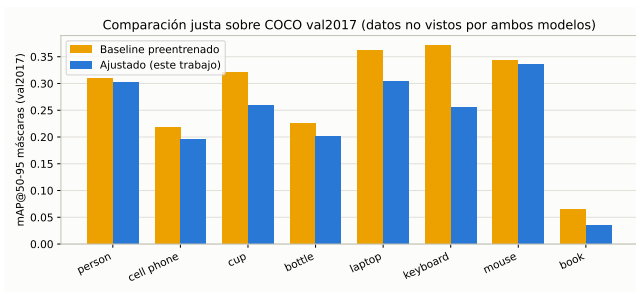
El baseline ya vio nuestro test

El modelo preentrenado (yolo_{v8n-seg}.pt) se entrenó sobre COCO **train2017**, el mismo conjunto del que muestreamos nuestras 5.000 imágenes. Es decir: el baseline fue entrenado sobre las mismas imágenes que aquí usamos como prueba, mientras que nuestro modelo se evalúa sobre datos genuinamente reservados.

La corrección: evaluar sobre COCO val2017

Se construyó una partición de prueba independiente desde **COCO val2017** —el conjunto de validación oficial, que el preentrenado nunca usó para ajustar sus pesos y es disjunto de train2017. Sobre esas 1.250 imágenes, *ninguno* de los dos modelos vio los datos: la comparación es limpia.

Comparación imparcial sobre COCO val2017



mAP@50-95	Base	Ajust.
Máscaras	0,277	0,236
Cajas	0,329	0,275

Al pasar a datos no vistos, el baseline **cae** de 0,342 a 0,277; nuestro modelo apenas varía (0,246 $\rightarrow$ 0,236).

Dos tercios de la brecha aparente eran la ventaja de fuga del baseline. Bajo la comparación justa hay **paridad** en person ($-0,008$) y mouse ($-0,007$).

Por qué el preentrenado es un baseline tan fuerte

Las 8 clases son un **subconjunto** de su vocabulario: las aprendió sobre 118.000 imágenes. Ajustar sobre 3.000 del mismo dominio no agrega información nueva; lo esperable es igualarlo, no superarlo en mAP.

El experimento que motivó la configuración final

Un ajuste fino **completo** (sin congelar capas, 60 épocas) sobre 3.000 imágenes quedó *por debajo* del modelo preentrenado sin tocar. La causa es la reinicialización de la cabeza de clasificación: 372.000 parámetros que deben reaprenderse con dos órdenes de magnitud menos de datos que en el preentrenamiento.

Configuración	mAP@50-95 cajas	mAP@50-95 máscaras
Ajuste fino completo, 60 épocas	0,250	0,216
Backbone congelado, 150 épocas	0,292	0,246

(métricas sobre la partición de prueba de train2017)

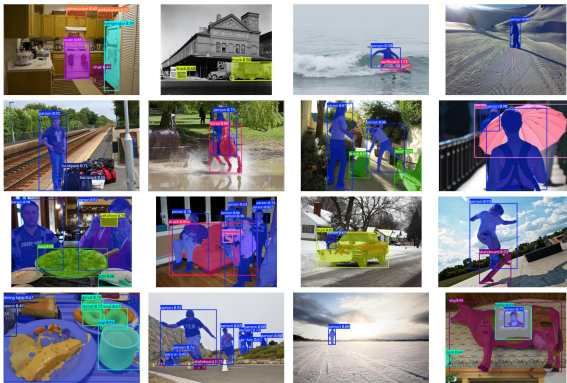
Lección

Cuando el vocabulario de destino es un **subconjunto** del de preentrenamiento, un ajuste fino ingenuo con la cabeza reinicializada puede *degradar* el modelo. Congelar el backbone recupera parte de la brecha y vuelve **segura** la transferencia.

Análisis cualitativo: el verdadero aporte del ajuste

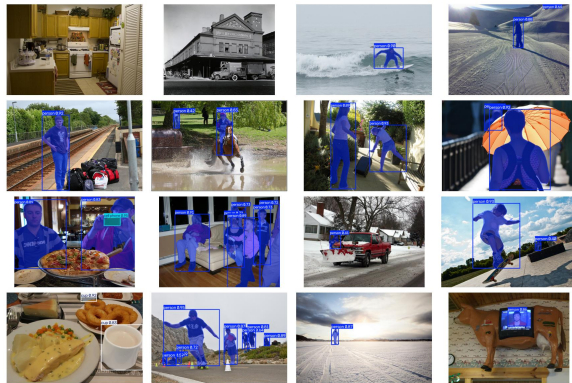
Baseline (80 clases)

Modelo preentrenado de referencia (80 clases) — COCO val2017



Ajustado (8 clases)

Modelo ajustado (8 clases, este trabajo) — COCO val2017



- El baseline dispersa detecciones fuera de alcance (refrigerator, surfboard, horse, umbrella, pizza, cow...).
- El ajustado se concentra **exclusivamente** en las 8 clases: suprime por completo los falsos positivos de vocabulario. *Esta* es su ventaja práctica, invisible al mAP por clase.

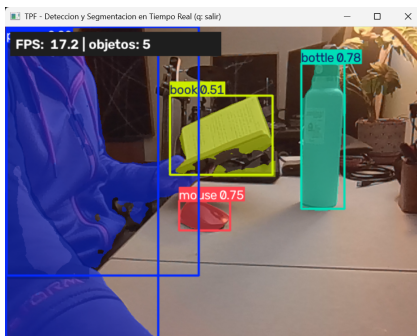
src/demo_webcam.py: captura por OpenCV, inferencia cuadro a cuadro y superposición de cajas, máscaras, etiquetas con confianza y FPS.

```
modelo = YOLO(args.model, task='segment')      # pesos .pt o modelo OpenVINO
captura = cv2.VideoCapture(fuente)             # webcam (índice) o archivo

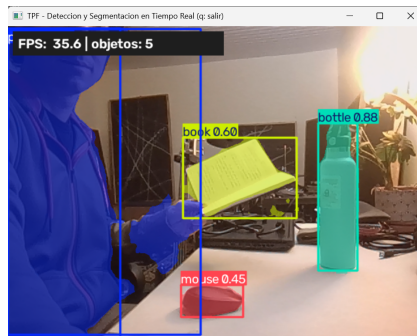
while True:
    ok, fotograma = captura.read()
    resultados = modelo.predict(fotograma, conf=args.conf, imgsz=args.imgsz)
    anotado = resultados[0].plot()              # cajas + máscaras + etiquetas
    cv2.imshow(ventana, anotado)
```

- FPS suavizados con media móvil exponencial (lectura estable).
- Fuente de video, umbral de confianza y resolución de inferencia configurables por línea de comandos.
- La computadora de demostración **no tiene GPU con CUDA** $\Rightarrow$ se exporta el modelo a **OpenVINO** (representación intermedia de Intel, especializada para CPU y gráficos integrados).

Plataforma	Formato	Latencia	FPS
Servidor, GPU RTX 5090	PyTorch (.pt)	5,1 ms/imagen	~198
PC personal, CPU Core Ultra 5	PyTorch (.pt)	~59 ms/cuadro	17
PC personal, CPU Core Ultra 5	OpenVINO	~29 ms/cuadro	35



PyTorch: 17,2 FPS



OpenVINO: 35,6 FPS

OpenVINO **duplica** la tasa de cuadros y supera el umbral de tiempo real (24–30 FPS) sin aceleración dedicada.

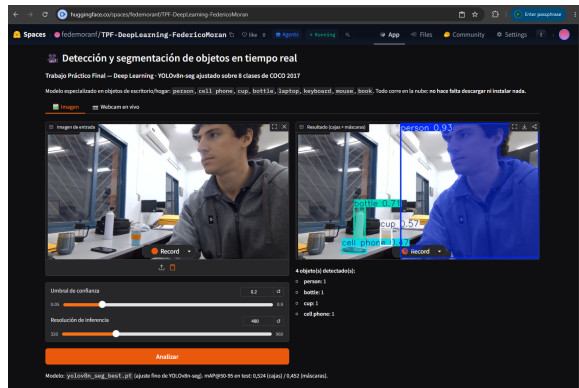
Despliegue en la nube: Hugging Face Spaces

La aplicación se desplegó en un **servidor público** con interfaz Gradio: quien la evalúe solo abre un enlace, sin descargar ni instalar nada.

Dos modos de uso

- **Imagen** — sube una fotografía; devuelve cajas, máscaras y recuento de objetos por clase.
- **Webcam en vivo** — inferencia cuadro a cuadro sobre la cámara del navegador, con umbral de confianza y resolución expuestos.

Los pesos viajan dentro del repositorio del Space: el modelo se ejecuta íntegramente en la nube.



Demostración en vivo

Detección y segmentación de instancias sobre cámara web

CPU Intel Core Ultra 5 226V — sin GPU dedicada

- **Desbalance de clases.** person concentra el 74,2 % de las instancias; mouse y keyboard tienen menos de 150 ejemplos de entrenamiento. No se corrigió de forma artificial; se tiene presente al interpretar las métricas por clase.
- **Calidad de la anotación de referencia.** La categoría book tiene límites ambiguos en COCO, lo que impone un techo a su desempeño medible.
- **Pérdida por simplificación de polígonos.** La conversión máscara $\rightarrow$ polígono (tolerancia 2 px) y el descarte de fragmentos degenerados eliminan ~ 4 % de las líneas de anotación.
- **Escala del conjunto de datos.** 3.000 imágenes de entrenamiento son dos órdenes de magnitud menos que el preentrenamiento de COCO; esa es precisamente la razón por la que congelar el backbone resulta determinante.

- ❶ **La evaluación rigurosa es tan determinante como el modelo.** Se corrigieron dos sesgos optimistas —la fuga en el propio *split* y el preentrenamiento del baseline sobre el mismo pozo de datos—; bajo una comparación imparcial en val2017 el modelo ajustado es competitivo con el preentrenado, y su aporte real es la especialización, no el mAP.
- ❷ **El desempeño por clase está gobernado por las características visuales y la calidad de anotación de cada categoría,** no por la confusión entre clases: la matriz de confusión muestra que el error dominante es la omisión de instancias pequeñas u ocluidas.
- ❸ **La segmentación de instancias en tiempo real es viable sobre una CPU de consumo.** La variante nano exportada a OpenVINO alcanzó 35 FPS sostenidos sin GPU dedicada, el doble que el formato PyTorch original.
- ❹ **El pipeline resultó reproducible de extremo a extremo,** con semillas fijas y controles de integridad que verifican, sobre los archivos exportados, que las particiones sean disjuntas.

- **Ampliar el conjunto de datos**, en particular para las clases raras (keyboard, mouse, laptop, cell phone), donde la reinicialización de la cabeza tiene mayor costo.
- **Explorar variantes de mayor capacidad** (YOLOv8s-seg) y cuantificar el intercambio entre precisión y latencia sobre la CPU de despliegue.
- **Atacar el recall** de instancias pequeñas: inferencia en mosaico (*tiling*), mayor resolución de entrada o aumento específico de escala.
- **Cuantización a INT8** con OpenVINO para elevar aún más la tasa de cuadros.

¿Preguntas?

Diapositivas de respaldo

¿Por qué YOLOv8 y no una arquitectura de dos etapas (Mask R-CNN)?

El requisito de tiempo real sobre CPU descarta las arquitecturas de dos etapas. YOLOv8-seg resuelve detección y segmentación en una sola pasada, con costo de máscara casi constante gracias a los prototipos.

¿Por qué la variante nano y no small o medium?

Es la única que sostiene tiempo real sobre la CPU de despliegue. La variante *small* quedó identificada como trabajo futuro, con la latencia como criterio de decisión.

¿Por qué AdamW y esa tasa de aprendizaje?

Selección automática de Ultralytics (`optimizer='auto'`), que fija $lr_0 = 8,33 \times 10^{-4}$ y momento 0,9 en función del tamaño del conjunto y del lote.

¿Por qué no se corrigió el desbalance con pesos de clase o focal loss?

El desbalance es la estadística natural del dominio y se traslada al escenario de despliegue. Corregirlo artificialmente optimizaría una distribución que no es la de uso.

¿Cómo se garantiza que no hay fuga de datos entre train y test?

Con dos controles independientes: (i) `untag_samples` elimina las etiquetas de split de origen de FiftyOne antes de particionar; (ii) un `assert` verifica, sobre los *archivos exportados* —que es lo que el entrenador realmente lee—, que las tres particiones sean disjuntas y midan 3.000 / 750 / 1.250.

¿Por qué la pérdida de validación estuvo por debajo de la de entrenamiento en corridas previas?

Era un síntoma. Con particiones correctas la pérdida de validación queda *por encima*, como corresponde. En una corrida anterior, val estaba contenida en train.

¿El aumento de datos se aplica también a las máscaras?

Sí. Ultralytics transforma imágenes y polígonos de forma consistente; *mosaic* recompone las cuatro máscaras en el lienzo combinado.

¿Por qué `close_mosaic`?

Mosaic produce imágenes de estadística artificial; `close_mosaic=10` lo desactiva en las últimas 10 épocas para afinar sobre imágenes naturales. En esta corrida el *early stopping* (época 140) se adelantó a esa fase, sin perjuicio para la convergencia.

¿Por qué el baseline se evalúa con 80 clases y el modelo ajustado con 8?

Porque son los vocabularios nativos de cada modelo; de las AP del baseline se extraen las 8 clases de interés sobre las mismas imágenes. Y —clave— la comparación principal se hace sobre COCO **val2017**, no sobre nuestra partición de train2017: el preentrenado fue entrenado sobre train2017, así que evaluarlo ahí lo favorece. Sobre val2017, no vista por ninguno de los dos, la comparación es limpia.

¿La partición de prueba se usó alguna vez para tomar decisiones?

No. El punto de control se selecciona por el mAP@50-95 de máscaras sobre **validación**. La prueba se evalúa una sola vez, al final.

¿Los FPS dependen del modelo entrenado?

No. La latencia depende de la arquitectura y del formato de ejecución, no de los valores de los pesos. Las mediciones de 17 y 35 FPS son estables entre corridas.

¿Qué pasa si la cámara falla en la demo?

Hay un video de respaldo: `python src/demo_webcam.py -source respaldo.mp4`.

Requisito	Dónde se cumple
Detección y/o segmentación	YOLOv8n-seg ajustado: cajas + máscaras
Preprocesamiento y <i>data augmentation</i>	Notebooks 01 y 02
Métricas estándar (mAP, IoU)	Notebook 03: mAP@50 y mAP@50-95
Inferencia en tiempo real	src/demo_webcam.py (FPS en pantalla)
Pipeline completo	Notebooks 01→03 + src/
Modelo entrenado	models/yolov8n_seg_best.pt
Reporte técnico con código por paso	reporte/informe.pdf
Demo en la nube, solo un enlace	Hugging Face Space (Gradio)
Presentación oral con demo	Este documento + demo en vivo